# Benchmarking Approximate Nearest Neighbour Search

**Shrawani Nanda** (2024101111)

Information Retrieval & Extraction - Assignment 3

**System & Libraries**: All experiments ran on a single Linux machine (12th Gen Intel Core i7-12650H, 10 cores / 16 threads, 16 GB RAM, x86_64). Software: FAISS 1.8.0 (`faiss-cpu`), NumPy 1.24.4, Python 3.10.16. Every search was run on a single CPU thread (`faiss.omp_set_num_threads(1)`), one query at a time, with latency recorded as the median over several runs on the evaluation split (queries 5,000-10,000). GloVe-100 vectors were $L_2$-normalised before indexing so that $\|u - v\|_2^2 = 2(1 - \cos\theta)$, letting us use Euclidean distance for all datasets.

## Part 1: Exact Baseline

The recall@10 function was written from scratch, taking care to handle ties correctly. Instead of simply matching neighbour IDs, we use a distance threshold, so a candidate at the same distance as the 10th true neighbour is not counted as a miss. Unit tests are also written covering cases such as a perfect match, a complete miss, ties at the boundary, and fewer than 10 neighbours. Exact search on the evaluation split matched the stored ground truth in both HDF5 files, with a recall@10 of 1.0000 on both datasets.

**Table 1: Part 1 - Exact Search Timings** (single-thread, evaluated on all 5,000 evaluation queries; queries 5,000-10,000).

| Dataset | Index | Recall@10 | Batch QPS | Batch Latency | Single QPS | Single Latency | Batch Speedup |
|---|---|---|---|---|---|---|---|
| **SIFT-128** | `IndexFlatL2` | 1.0000 | 67.74 | 14.76 ms | 23.71 | 42.17 ms | **2.86**× |
| **GloVe-100** | `IndexFlatL2` | 1.0000 | 72.54 | 13.79 ms | 26.67 | 37.49 ms | **2.72**× |

**Why Single-Query and Batch Differ**: When processing queries one at a time, each query requires a separate matrix-vector multiplication (GEMV). This means the entire database (around 500 MB) has to be read from RAM for every query, making memory bandwidth the main bottleneck. When all queries are processed together, the operation becomes a matrix-matrix multiplication (GEMM). Database blocks can then be reused across multiple queries instead of being loaded repeatedly. This makes better use of the CPU's AVX2/FMA units and reduces memory transfer overhead. Python function calls and memory allocation overhead are also reduced because they are handled once for the entire batch. Together, these factors explain the 2.72–2.86× improvement in throughput.

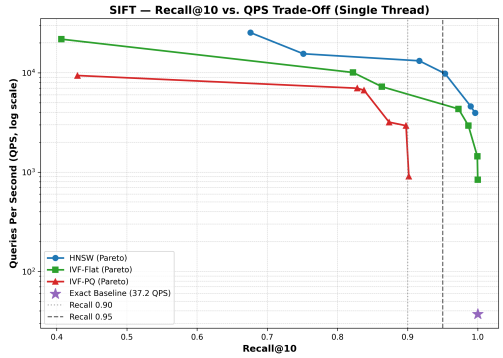
## Part 2: Benchmark Three Indexes (HNSW, IVF-Flat, IVF-PQ)

For each index, several build configurations were tried and swept the corresponding search-time parameters. For HNSW, I tested $M \in \{8, 16, 32\}$ with `efConstruction` = 64, sweeping `efSearch` $\in \{8, 16, 32, 64, 128, 256, 512\}$. For IVF-Flat, I tested `nlist` $\in \{512, 1024, 2048, 4096\}$ and swept `nprobe` $\in \{1, 2, 4, 8, 16, 32, 64, 128, 256\}$. For IVF-PQ, I tested `nlist` $\in \{512, 1024\}$, with $M_{\text{PQ}} \in \{16, 32, 64\}$ for SIFT and $M_{\text{PQ}} \in \{10, 20, 25, 50\}$ for GloVe, using the same `nprobe` sweep as IVF-Flat. These configurations covered recall values from below 0.5 up to the maximum achievable by each index. Then the Pareto frontier was extracted, representing the best QPS at each recall level. All timings were measured using single-threaded, single-query search, with the median taken across repeated runs.

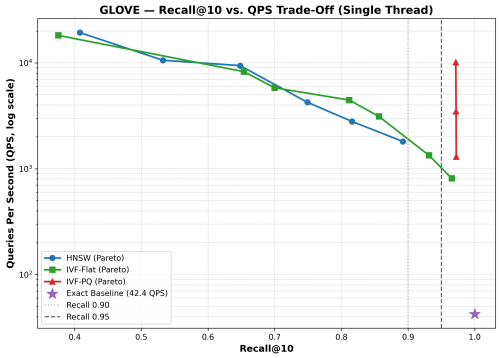
**Table 2: Part 2 - Throughput at Fixed Recall Targets** (single-thread, single-query, queries 5,000-10,000; QPS log-linearly interpolated along Pareto frontier).

| Dataset | Index | QPS @ 0.90 | QPS @ 0.95 | Max Recall | Exact QPS | Speedup vs. Exact @ 0.95 |
|---|---|---|---|---|---|---|
| **SIFT-128** | **HNSW** | **10,603.2** | **7,412.5** | 0.9992 | 37.23 | **199.1**× |
| | **IVF-Flat** | 4,158.4 | 1,948.0 | 1.0000 | 37.23 | 52.3× |
| | **IVF-PQ** | 2,765.5 | *N/A (< 0.95)* | 0.9323 | 37.23 | *N/A* |
| **GloVe-100** | **HNSW** | 1,174.5 | *N/A (< 0.95)* | 0.9262 | 42.36 | *N/A* |
| | **IVF-Flat** | 934.9 | **367.6** | 0.9806 | 42.36 | **8.7**× |
| | **IVF-PQ** | **23,560.8** | 269.9 | 0.9551 | 42.36 | 6.4× |

**Why Some Indexes Fail to Reach High Recall**:




**(a)** SIFT-128 ($N = 1$M, $D = 128$, Euclidean).




**(b)** GloVe-100 ($N = 1.18$M, $D = 100$, Angular/$L_2$).

**Figure 1: Part 2 - Recall@10 vs. Throughput Pareto Curves** on SIFT-128 and GloVe-100. Each point is a (build config, `efSearch`/`nprobe`) pair; only the Pareto frontier is shown per index. Dashed black lines indicate the single-query exact brute-force baseline.

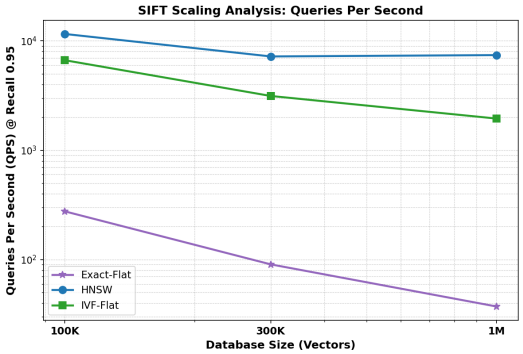
**Table 3: Part 2 - Build Cost & Memory at Recall $\geq$ 0.95** (best Pareto config per index; build uses 16 threads; search uses 1 thread).

| Dataset | Index | Config @ $\geq$0.95 | Recall | Build (s) | Size (MB) | Bytes/vec | Compression |
|---|---|---|---|---|---|---|---|
| **SIFT-128** | **HNSW** | $M$=32, efC=64, efS=64 | 0.9633 | 37.42 | 747.8 | 784.1 | 1.53× expansion |
| | **IVF-Flat** | nlist=4096, nprobe=64 | 0.9768 | 310.12 | 497.9 | 522.1 | 1.02× raw |
| | **IVF-PQ** | *max achieved ($M_{PQ}$=64, np=256)* | 0.9323 | 29.90 | 69.3 | 72.7 | 7.05× compressed |
| **GloVe-100** | **HNSW** | *max achieved ($M$=32, efS=512)* | 0.9262 | 59.37 | 758.7 | 672.2 | 1.68× expansion |
| | **IVF-Flat** | nlist=1024, nprobe=128 | 0.9511 | 23.11 | 460.9 | 408.4 | 1.02× raw |
| | **IVF-PQ** | nlist=1024, $M_{PQ}$=50, np=256 | 0.9551 | 30.13 | 66.0 | 58.4 | **6.85× compressed** |

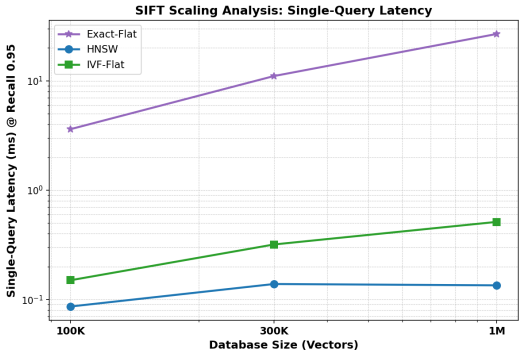
- **SIFT-128 / IVF-PQ** (max recall 0.9323): Product Quantization represents each sub-vector using its nearest codebook centroid, which introduces quantization error $\epsilon_{\text{quant}} = \|x - \hat{x}\|$. Reducing this error requires longer codes. In SIFT, the distance difference between the true 10th neighbour and the 11th is often small. As a result, even a small quantization error can change their relative ranking, causing some true neighbours to be missed. This limitation remains even when 25% of the inverted lists are searched (nprobe = 256).
- **GloVe-100 / HNSW** (max recall 0.9262): Normalised GloVe vectors lie on a 100-dimensional unit sphere and exhibit a high degree of hubness, where a few vectors appear in the neighbourhoods of many queries. During graph search, the greedy beam search used by HNSW can get trapped in local minima. With $M \leq 32$, the graph may not provide enough connections to reach some true neighbours, even when they are close to the query. This limits the maximum achievable recall.

# Part 3: Scaling Experiment on SIFT (100K, 300K, 1M)

**Hypothesis**: *"As the database grows from 100K to 300K to 1M vectors, exact search will require increasingly more computation per query. HNSW and IVF-Flat may provide better query-time scaling than exact search because they can avoid exhaustively comparing every database vector. Maintaining Recall@10 = 0.95 may require increased search effort as the database grows."*




**(a)** Throughput (QPS) vs. database scale $N$.




**(b)** Latency (ms/query) vs. database scale $N$.

**Figure 2: Part 3 - SIFT Scaling Dynamics**: QPS and latency across $N \in \{100\text{K}, 300\text{K}, 1\text{M}\}$ at Recall@10 $\geq 0.95$.

**Table 4: Part 3 - SIFT Scaling at Recall ≥ 0.95** (single-threaded search, queries 5,000-10,000).

| Scale ($N$) | Index | Config | Recall | QPS | Latency | Build (s) | Size (MB) | Speedup vs. Exact |
|---|---|---|---|---|---|---|---|---|
| **100K** | **Exact-Flat** | Brute-force reference | 1.0000 | 276.0 | 3.62 ms | 0.03 | 48.8 | 1.0× |
| | **HNSW** | $M$=8, efC=64, efS=64 | 0.9622 | **11,570.2** | 0.09 ms | 1.53 | 56.5 | **41.9**× |
| | **IVF-Flat** | nlist=1024, nprobe=32 | 0.9652 | 6,655.6 | 0.15 ms | 18.33 | 50.1 | 24.1× |
| **300K** | **Exact-Flat** | Brute-force reference | 1.0000 | 90.4 | 11.07 ms | 0.11 | 146.5 | 1.0× |
| | **HNSW** | $M$=16, efC=64, efS=64 | 0.9675 | **7,207.2** | 0.14 ms | 8.11 | 187.8 | **79.7**× |
| | **IVF-Flat** | nlist=512, nprobe=16 | 0.9562 | 3,134.9 | 0.32 ms | 12.34 | 149.0 | 34.7× |
| **1M** | **Exact-Flat** | Brute-force reference | 1.0000 | 37.2 | 26.86 ms | 0.18 | 488.3 | 1.0× |
| | **HNSW** | $M$=32, efC=64, efS=64 | 0.9633 | **7,412.5** | 0.13 ms | 37.42 | 747.8 | **199.1**× |
| | **IVF-Flat** | nlist=4096, nprobe=64 | 0.9768 | 1,948.0 | 0.51 ms | 310.12 | 497.9 | 52.3× |

**Did the Results Support the Hypothesis?**

The results were consistent with all three predictions:

- **Exact search scales as $\mathcal{O}(N)$ (Fully Supported)**: The time taken per query increased linearly (3.62 → 11.07 → 26.86 ms), as expected since exact search has to scan the entire database for every query.
- **ANN query scaling is sub-linear (Fully Supported)**: HNSW latency remained almost constant (0.09 → 0.14 → 0.13 ms) even as the dataset size increased by 10×, which is consistent with $\mathcal{O}(\log N)$ graph traversal. Its speed advantage over exact search also increased from 41.9× at 100K vectors to 199.1× at 1M vectors. IVF-Flat showed roughly $\mathcal{O}(\sqrt{N})$ scaling, with latency increasing from 0.15 to 0.51 ms. Its speed advantage over exact search grew from 24.1× to 52.3×.
- **More search effort needed as $N$ grows (Partially Supported)**: To achieve a recall of at least 0.95, I had to increase the graph degree $M$ (8 → 16 → 32), rather than increase `efSearch`. An unexpected observation was the increase in build time: HNSW build time grew by 24.5×, while IVF-Flat build time increased sharply to 310 s. This was mainly because $k$-means had to find and converge 4,096 centroids over one million vectors.

# Part 4: Own-Data Experiment - Hard Geometry (Cluster Separation)

**Hypothesis**: *"As clusters become less separated and overlap increases, IVF-Flat require a higher* `nprobe` *to achieve Recall@10 = 0.95, since relevant neighbours may lie in clusters that are not probed. HNSW may exhibit a different sensitivity to cluster overlap because it searches a graph rather than selected clusters."*

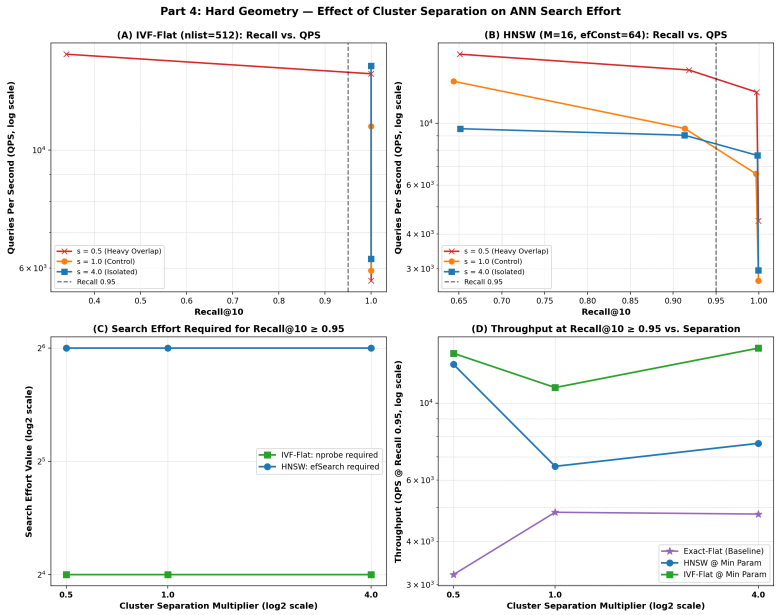



**Figure 3: Part 4 - Hard Geometry Main Figure**: Effect of cluster separation $s \in \{0.5, 1.0, 2.0, 4.0\}$ on HNSW and IVF-Flat. Top-left: Recall vs. QPS Pareto curves. Top-right: Minimum search-time parameter to reach Recall@10 $\geq 0.95$. Bottom-left: Parameter sweep sensitivity. Bottom-right: Single-query latency across separation settings.

**Setup**: We generated synthetic Gaussian mixture data ($D = 64$, $K = 20$ clusters, $N = 100{,}000$ per setting, seed 42) with four separation multipliers $s \in \{0.5, 1.0, 2.0, 4.0\}$, using $s = 1.0$ as the baseline. Centroids $\mu_k \sim \mathcal{N}(0, I_D)$, cluster assignments $c_i$, and noise $\epsilon_i \sim \mathcal{N}(0, I_D)$ were kept fixed across all settings; only the scale in $x_i(s) = s \cdot \mu_{c_i} + \epsilon_i$ changed. This keeps cluster separation as the only thing that varies.

**Table 5: Part 4 - Cluster Geometry Diagnostics & Performance at Recall ≥ 0.95.**

| Sep. ($s$) | Regime | Inter / Within Spread | Ratio | Index | Min Param @ 0.95 | Recall | QPS | Speedup |
|---|---|---|---|---|---|---|---|---|
| $s = 0.5$ | Severe Overlap | 17.01 / 7.97 | 2.13 | Exact-Flat | Brute-force ref | 1.0000 | 321.1 | 1.0× |
| | | | | **HNSW** | `efSearch` = 96 | 0.9602 | 4,000.6 | 12.5× |
| | | | | **IVF-Flat** | `nprobe` = 20 | 0.9503 | **6,302.1** | **19.6×** |
| $s = 1.0$ | Control | 34.02 / 7.97 | 4.27 | Exact-Flat | Brute-force ref | 1.0000 | 448.4 | 1.0× |
| | | | | **HNSW** | `efSearch` = 80 | 0.9541 | 4,509.1 | 10.1× |
| | | | | **IVF-Flat** | `nprobe` = 20 | 0.9502 | **6,670.5** | **14.9×** |
| $s = 2.0$ | Well-Separated | 68.03 / 7.97 | 8.54 | Exact-Flat | Brute-force ref | 1.0000 | 359.9 | 1.0× |
| | | | | **HNSW** | `efSearch` = 80 | 0.9577 | 3,080.4 | 8.6× |
| | | | | **IVF-Flat** | `nprobe` = 20 | 0.9517 | **4,518.9** | **12.6×** |
| $s = 4.0$ | Isolated Islands | 136.07 / 7.97 | 17.08 | Exact-Flat | Brute-force ref | 1.0000 | 374.8 | 1.0× |
| | | | | **HNSW** | `efSearch` = 80 | 0.9582 | 4,222.7 | 11.3× |
| | | | | **IVF-Flat** | `nprobe` = 20 | 0.9539 | **4,432.3** | **11.8×** |

**What Happened & Why**:

1. **IVF-Flat hypothesis refuted**: The minimum `nprobe` required to achieve a recall of at least 0.95 was **20 across all settings**. This is because Lloyd's $k$-means algorithm places centroids based on the data density. Therefore, the within-cluster spread ($\sigma = 1.0$), which remained fixed, matters more than the distance between cluster centres. As a result, probing 20 out of 512 cells was enough to cover approximately 78% of a query's local volume, regardless of the overall cluster separation.
2. **HNSW weakly consistent**: When the clusters overlapped more ($s = 0.5$), a slightly higher search effort was needed, with `efSearch`=96 compared to 80 for the other settings. The overlapping clusters introduce shortcut edges in the graph, which can sometimes lead the greedy search towards the wrong cluster. However, the difference was quite small in practice: even at `efSearch`=80, recall for $s = 0.5$ was already 0.9468, just 0.0032 below the target.
3. **Untested**: I did not explore the effects of changing the cluster shape (anisotropic covariance), the number of clusters $K$, the within-cluster intrinsic dimension, or the extreme case where the clusters completely collapse ($s \to 0$).

# Part 5: Synthesis & Discussion

**Q1: At recall 0.95, which index is fastest on each dataset, and does the answer change between datasets?**

- **SIFT-128**: **HNSW** is the fastest, achieving 7,412.5 QPS (0.13 ms per query). It is 3.8× faster than IVF-Flat (1,948 QPS) and 199× faster than exact search. IVF-PQ does not reach 0.95 recall on this dataset.
- **GloVe-100**: **IVF-Flat** performs best at 367.6 QPS (2.72 ms per query), making it 8.7× faster than exact search. IVF-PQ also reaches 0.95 recall, but at a lower speed of 269.9 QPS. HNSW reaches a maximum recall of 0.926.

The fastest index is different between the two datasets. SIFT descriptors have relatively low intrinsic dimensionality and clear spatial structure, which allows HNSW's multi-layer graph to navigate the data efficiently. GloVe embeddings, on the other hand, lie on a unit hypersphere and exhibit hubness, where a few vectors appear in many queries' neighbourhoods. HNSW's greedy search can struggle to navigate around these hubs, and a graph with $M \leq 32$ connections may not be sufficient to reach some true neighbours. IVF-Flat handles this differently by directly scanning the raw vectors within the selected Voronoi cells, which allows it to achieve higher recall on this dataset.

**Q2: What do the indexes cost in build time and memory? What does PQ save, and what does it give up?**

- **Build time**: HNSW builds relatively quickly at larger dataset sizes (37 s on SIFT-1M and 59 s on GloVe, using 16 threads), since inserting each vector takes $\mathcal{O}(\log N)$ time. IVF-Flat takes longer at scale: training $k$-means with 4,096 centroids on 1M vectors took 310 s. IVF-PQ takes about 30 s to train both the coarse centroids and sub-quantizer codebooks. This is faster than IVF-Flat because it uses a sample of the data for training rather than the entire dataset.
- **Memory**: HNSW stores the original vectors along with adjacency lists, resulting in 1.53-1.68× the memory required by the raw data. IVF-Flat has very little additional overhead, using about 1.02× the raw data size. IVF-PQ uses the least memory, requiring only 72.7 bytes per vector on SIFT (7.05× compression) and 58.4 bytes per vector on GloVe (6.85× compression). This makes it possible to store much larger databases in memory and potentially benefit from CPU cache.
- **What PQ gives up**: PQ replaces each sub-vector with its nearest codebook centroid, introducing a quantization error $\epsilon_{\text{quant}} = \|x - \hat{x}\|$. This error cannot be completely eliminated while keeping the code size fixed. When the distance gap between the true 10th and 11th nearest neighbours is small, quantization can change their relative ranking and reduce

recall. Recovering the lost accuracy may require re-ranking candidates using the original vectors.

**Q3: What did your experiments in Parts 3 and 4 show, and what do they imply about when the headline numbers could mislead?**

1. **QPS does not capture construction cost**: HNSW query latency grows as $\mathcal{O}(\log N)$, but its build time increased by 24.5× when the dataset size grew by 10×. This matters in applications where the index needs to be rebuilt or updated frequently, even if its query performance looks impressive.
2. **Performance on one dataset does not guarantee similar results on another**: HNSW achieves a recall of 0.999 on SIFT but only 0.926 on GloVe with the same settings. This shows why reporting recall on just one dataset can give a misleading picture of an index's performance.
3. **Batch QPS can overstate single-query performance**: Processing queries in batches improved throughput by 2.72-2.86× due to BLAS-3 operations and cache reuse (Part 1). Reporting batch throughput for an interactive search system would therefore give an overly optimistic estimate of its actual performance.
4. **Clustered benchmarks may hide certain failure cases**: Part 4 showed that $k$-means adapts its centroids to the cluster structure, which meant that the same `nprobe`=20 was sufficient across all four separation settings. However, this does not tell us how the index would perform on uniform or high-dimensional data distributions, which were not tested.

**Q4: Suppose you have 150,000 vectors, or 10 million, a 100 ms budget per query and 16 GB of RAM. Using your exact-search timings and your results, would you use ANN, and if so which index and settings?**

- $N = 150{,}000$ **($D = 128$, float32)**: The raw vectors require around $150{,}000 \times 512\,\text{B} = 76.8\,\text{MB}$, which is well under 1% of the available RAM. Based on our 100K measurement (3.62 ms), exact search should take around 5.4 ms per query at this size, which is about 18 times faster than the 100 ms limit. Therefore, I would use exact search (`IndexFlatL2`). It provides perfect recall without requiring any index construction or parameter tuning.
- $N = 10{,}000{,}000$ **($D = 128$, float32)**: Exact search would take approximately 268.6 ms per query, based on linear scaling from the 1M measurement. This exceeds the 100 ms budget by about 2.7×, so ANN becomes necessary. I would use HNSW with $M = 16$, efConstruction = 128, and efSearch = 64. The index is expected to require around 7.8 GB of memory ($5.12 \times 1.53$), which fits within the 16 GB limit. Expected query latency is under 0.3 ms, with recall above 0.95. If memory is limited because of other running services, IVF-PQ with re-ranking (`nlist`=16384, $M_{\text{PQ}} = 32$) is another option. It requires only around 700 MB and is expected to stay under 5 ms per query.

**Q5: Name two ways your own benchmark could be misleading, and say whether you tested them.**

1. **Single-threaded vs. concurrent queries**: All timings were measured using a single thread (`faiss.omp_set_num_threads(1)`). Real-world servers often handle multiple queries at the same time. This can lead to memory bandwidth contention for IVF-Flat and exact search, while HNSW may experience L3 cache contention due to the poor memory locality of graph traversal. As a result, the relative performance of the indexes could change under concurrent workloads. *Not tested, since the assignment requires single-threaded timing.*
2. **Uniform vs. skewed or out-of-distribution queries**: Our test queries were drawn from the same distribution as the indexed data. In practice, query distributions may be skewed (for example, following a Zipf distribution), arrive in bursts, or shift over time. Such changes could affect HNSW graph traversal and make the IVF partitions less suitable for the incoming queries. *Partially explored in Part 4 through the cluster-separation experiments, but dynamic updates and skewed query distributions were not tested.*